Transport Supply Update Process

The Voyager Converter v2.1 imports rail and bus timetable data and uses Cube network information to build rail and bus routes to create the .LIN PT lines files. This is done using a Python tool with GUI to load in the relevant files. Key inputs are:

- Bus stop information
- Rail station information
- Rail Rolling Stock information
- Bus and Rail timetable information
- Bus and Rail Operator information
- Mode information

Inputs

Bus Stops

- Input: Node Lookup Files\naptan_to_node_lookup.csv
- All GB stop information downloaded from https://beta-naptan.dft.gov.uk/Download/National
- Import this file to QGIS.
- Import the network nodes to QGIS, ensuring to filter out zone nodes, rail station nodes, and ferry port nodes.
- In the Processing Toolbox, use the "Distance to nearest Hub (points)" tool to associate the nearest bus stop to a network node.
- After the tool has run, it is worth spending time to refine the naptan_to_node equivalence as this will greatly help the route patching process. Some points to bear in mind include:
  - ensure bus stops are not assigned to the wrong side of the carriageway in dual carriageway situations
  - assign all bus stops associated with a bus station to the most appropriate single network node

Rail Stations

- Input: Node Lookup Files\tmfs_tiploc_to_node.csv
- The previous *tmfs_tiploc_to_node.csv* was used because station information does not generally change over time. The Robroyston node number needed updated. New stations were appended to the end of this list using the station info downloaded below (the .MSN file). New stations added were:
  - Kintore
  - Leven
  - Cameron Bridge
  - East Linton
  - Reston
  - Inverness Airport

Rail Rolling Stock

- Inputs: Node Lookup Files\rolling_stock_stations_lookup.csv
  - Node Lookup Files\rolling_stock_data.csv
- Used to calculate the capacity on rail services. The previous *rolling_stock_stations_lookup.csv* was used and the above stations appended to the end of the list of the list.
- New rolling stock data was provided by Transport Scotland derived from Bluetooth information. A new list of rail services by Origin and Destination was derived from the newly imported timetable data and the Bluetooth data used to derive new Seat and Crush capacities for each service.

Rail Data

- Input: Data\NationalRail\RJTTF386.MCA and RJTTF386.MSN
- **Open Rail Data**
  - .MCA (timetables) and .MSN (station info) files are required to import the timetable information through the GUI. (not just a CIF file)
  - This was obtained via https://wiki.openraildata.com/index.php/DTD and the National Rail Data Portal https://opendata.nationalrail.co.uk/. User registration is required but free and quick to access.
  - The data was download using a free tool called Postman that lets you build requests to fetch the data.
  - Data is "on the day", so no historical timetable information is available.

Bus Data

- Input: Data\NCSD_TXC and TransXchange folders
- **TransXChange**
  - .XML format files obtained through Traveline. Free account registration required. https://www.travelinedata.org.uk/traveline-open-data/traveline-national-dataset/
  - This downloaded an "S" folder for all services within Scotland
  - Data is "on the day", so no historical timetable information is available.
- **NCSD – long distance services**
  - NCSD data obtained from Datacutter. https://basemap.co.uk/datacutter
  - This download included .XML format files for operators Flixbus, Mega/Scottish Citylink, and National Express.

Operators

- Input: Node Lookup Files\TMfS18 Operator Lookup.csv
- This file can be self-generated by leaving it out of the "Node Lookup Files" folder or changing the name from the above. However, not all operators (rail and bus) will be populated based on a single import of, for example, just the Scottish TransXChange data. All three data types (rail, txc, ncsd) would need imported generating three separate lookup files.
- To self-generate the file the operators are looked up in the .xml files either by <Operator id> or <OperatorCode>, it's not entirely clear why some seem to be found using one over the other.

- Stagecoach seems to be a particularly tricky operator, as the self-generated file does not pull in Stagecoach West Scotland to the TMfS18 Operator Lookup.csv. Checking the actual .xml files for the potential codes used, whether Alpha or Numeric, and including both in the lookup list, then re-importing the data, should then catch all potential codes.
- The TMfS18 Operator Lookup.csv file needs to be built carefully so that the operator numbers assigned will ultimately tie up with the System.PTS file which is an input to the Cube model.

Mode

- Input: Node Lookup Files\TMfS18 Mode Lookup.csv
- Left unchanged from previous work.

Voyager Converter Tool Process

Install all the packages required in Voyager_Converter_v2.1\requirements.txt. This list has not been updated, installing the latest version of each is acceptable. Run *Main.py* to launch the GUI.

Default Variables

- Input: Voyager_Converter_v2.1\default_variables.txt
- This file can be updated to reference the necessary default files to be loaded into the GUI.

Intermediate folder

- All outputs* from the process are written to a folder Voyager_Converter_v2.1\Intermediate. They will be written over with each subsequent run of the GUI. Change the name of the Intermediate folder if you wish to keep previous outputs (e.g. between rail/txc/ncsd runs).
- * a handful of outputs are written to the top level Voyager_Converter_v2.1 folder, and this has not yet been changed during the update to the tool. These will also be written over with each subsequent run of the GUI so need copied into the relevant Intermediate folder if they are to be kept. They are:
  - flagged_bus_routes.csv
  - mode_lookup.csv
  - unused_xml_files.txt
  - headways_running_times.csv

GUI and Outputs

General Options Tab

- Fairly self-explanatory by text on screen. 2025 update has used Wednesday, date range of 1-31 March 2025, and default files as included in the default_varliables.txt.

Bus Import Tab

- Step 1: Import Data
  - Log window at the bottom of the screen will detail any .xml files which had errors when importing. Copy this text and save to a file for review.
  - Outputs:
    - **headways.csv** – key file detailing all services with known nodes and departure times converted to periodic headways.
    - **individual_routes.csv** – all services by individual departure time
    - **headways_summary.csv** – less detail than headways.csv
- Step 2: Filter Operators
  - Easiest to "Add all" and press Filter Operators button
  - Output:
    - **headways_filtered_operators.csv** – same file as *headways.csv* unless the "TMfS18 Operator Lookup.csv" is missing operators.
- Step 3: Open Patching Interface
  - Ensure node and link files are correct.
  - *headways_filtered_operators.csv* is used as the basis for the patching process.
    - CA25_roadnodes.csv – export nodes.dbf from Cube, remove zone nodes, rail station nodes, and ferry port nodes.
    - CA25_roadlinkstimes.csv - Export links.dbf from Cube, keep A, B, Link_Type, Distance, One_Way, Speed. Calculate the free flow link Time(mins) based on speed and distance. The Header of the csv must still say Distance and not Time, because the script is looking for the word Distance, but patching using Time yields a better result. Remove from the file all zone nodes below 1000, and greater than and equal to 100000. Also remove any links for Edinburgh Tram only, to prevent buses routing onto them, so remove links using nodes 80002, 80003, 80008-80022.
  - *Patching_overrides.txt* can be created manually to enforce a strict path between a pair of bus stops. For a long route, each pair of bus stops in the sequence would need input.
  - The 2025 update has followed two processes:
    - Patch shorter distance services, ie: those which tend to have many bus stops closer together, with the "Use Distance" box ticked on. This calculates the route between bus stops based on checking a set number of nodes (500). This is a more data intensive process and took 18 hours to complete.
    - Patch longer distance services, ie: cross country services with only 3 or 4 stops, with the "Use Distance" box ticked off. This calculates the shortest routes based on the number of nodes passed. For long distance coaches sticking to motorways, this was an acceptable approach and patched in approximately 30min.
  - Output: **headways_patched.csv**
- Step 4: Print LIN File
  - *headways_patched.csv* is used as the basis to print a LIN file.

- The resulting file is likely to be unreadable by Cube due to very long LONGNAMEs which fall over two rows. A secondary script has been developed to quickly rewrite the file ensuring the LONGNAME stays on one row.
- Run *LIN_Converter_v0.1.py*

Rail Import Tab

- As above but with rail specific files. Rail nodes and Rail links .csv files can also be generated by exporting the .dbf files from the Cube network. Include only nodes and links in the 100000s range.

Refinement to the Process

The starter procedure should be to patch all routes with "Use Distance" ticked on. This will likely result in services showing up with longer segments on the network as "straight lines" because a route could not be found between bus stops. A list of services needing re-patched with a different method (either checking a higher number of nodes or "Use Distance" ticked off) will need to be generated.

The Intermediate folder only needs the file *headways_filtered_operators.csv* to start the patching process again. However, editing this .csv in Excel and saving it results in the formatting of some cells going "wrong" for some services and being unusable by the tool.

Headways Formatter

- Run *Headways Formatter.py,* which opens a small GUI window to drag and drop the specified file into.
- Drag in *headways_filtered_operators.csv* and the script will reformat it to an .xlsx. It separates the headways into individual columns and calculates the current distance of the route (though this is now Link Time as that is what is used in the 2025 update).
- Services can be filtered in any way required in Excel.
- Update the file *lines_to_remove.csv* which sits beside Headways Formatter.py with a list of services to remove given by their "Line_Name". For example, if there are 50 services which need re-patched, the **opposite** list of Line_Names needs put in the *lines_to_remove.csv* file.
- Re-run the script and click the button to create a "lines removed" version of the *headways_filtered_operators.csv* file without errors, leaving only the 50 services needing re-patched. The script always needs fully closed and re-run if loading in a new *lines_to_remove.csv,* as it only "reads" this in the first time the window opens.
- Similarly, if no extra patching is required, drag and drop in *headways_patched.csv* to produce the Excel version. In Excel, filter out any operators/services that will not be kept in the final network. Create another list for a new *lines_to_remove.csv* of all services not being kept in the network.
- Re-Open and re-run the script to create a "lines removed" version of *headways_patched.csv.*

- It is this version of *headways_patched.csv* that needs to be in an Intermediate folder (it could be the only file), then skip to Step 4 and print the finalised LIN file with just those services to be kept in the network.
- Again, run *LIN_Converter_v0.1.py* if the LONGNAMES are falling over two rows.